Здесь будет титульник, листай ниже

СОДЕРЖАНИЕ

1 ПОСТАНОВКА ЗАДАЧИ.....5
1.1 Описание входных данных.....7
1.2 Описание выходных данных.....7
2 МЕТОД РЕШЕНИЯ.....9
3 ОПИСАНИЕ АЛГОРИТМОВ.....12
3.1 Алгоритм конструктора класса NetworkDevice.....12
3.2 Алгоритм метода setSpeed класса NetworkDevice.....12
3.3 Алгоритм метода getSpeed класса NetworkDevice.....12
3.4 Алгоритм конструктора класса NetworkStorage.....13
3.5 Алгоритм метода getName класса NetworkStorage.....13
3.6 Алгоритм метода printInfo класса NetworkStorage.....14
3.7 Алгоритм конструктора класса StorageDevice.....14
3.8 Алгоритм метода SetCapacity класса StorageDevice.....14
3.9 Алгоритм метода getCapacity класса StorageDevice.....15
3.10 Алгоритм функции main.....15
4 БЛОК-СХЕМЫ АЛГОРИТМОВ.....19
5 КОД ПРОГРАММЫ.....25
5.1 Файл main.cpp.....25
5.2 Файл NetworkDevice.cpp.....26
5.3 Файл NetworkDevice.h.....27
5.4 Файл NetworkStorage.cpp.....27
5.5 Файл NetworkStorage.h.....27
5.6 Файл StorageDevice.cpp.....28
5.7 Файл StorageDevice.h.....28
6 ТЕСТИРОВАНИЕ.....30
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....31

1 ПОСТАНОВКА ЗАДАЧИ

В системе управления цифровыми устройствами используются два типа объектов. Первый тип описывает сетевые устройства, обладающие параметром скорости соединения. Второй тип описывает устройства хранения данных, обладающие объёмом памяти.

Некоторые устройства объединяют свойства обоих типов. Такие устройства называются сетевыми хранилищами. Они могут одновременно работать в сети и хранить данные. Для моделирования такой системы используются:

- класс сетевого устройства;
- класс устройства хранения данных;
- класс сетевого хранилища, который наследует свойства обоих классов.

Требуется разработать систему, которая

- демонстрирует использование множественного наследования;
- демонстрирует создание объектов производного класса;
- демонстрирует хранение указателей на объекты в контейнере vector;
- демонстрирует обработку нулевого указателя;
- демонстрирует изменение объектов контейнера по заданному условию.

Класс сетевого устройства

- В закрытом разделе:
    - целочисленное свойство скорости сети.

В открытом разделе

- параметризированный конструктор;
- метод изменения скорости;
- метод получения скорости.

Класс устройства хранения данных

В закрытом разделе

- целочисленное свойство объёма памяти.

В открытом разделе

- параметризированный конструктор;
- метод изменения объёма памяти;
- метод получения объёма памяти.

Класс сетевого хранилища должен:

- наследоваться от двух базовых классов;
- содержать строковое свойство имени устройства;
- иметь конструктор с параметрами:
  - имя устройства;
  - скорость сети;
  - объём памяти;
- иметь метод вывода информации об объекте. Метод вывода должен отображать: <имя> <скорость> <память>

Алгоритм работы программы

1. Ввести количество устройств.
2. Объявить контейнер указателей на объекты сетевого хранилища.
3. Для каждого устройства ввести: <имя> <скорость> <память>, динамически создать объект сетевого хранилища и сохранить указатель на него в контейнере.
4. После ввода всех устройств добавить в контейнер один нулевой указатель.
5. Вывести строку Initial state
6. Вывести информацию обо всех элементах контейнера:
   6.1. если указатель не равен nullptr, вывести информацию об

устройстве.

6.2. если указатель равен, вывести Empty pointer.

7. Ввести значение минимально допустимой скорости сети.

8. Для каждого элемента контейнера:

8.1. если указатель равен nullptr, перейти к следующему элементу;

8.2. если скорость устройства меньше введённого значения:

8.2.1. удалить старый объект;

8.2.2. создать новый объект с тем же именем и объёмом памяти;

8.2.3. установить скорость сети равной 2 * введённое значение;

8.2.4. сохранить указатель на новый объект в контейнере.

9. Вывести строку Modified state

10. Вывести информацию обо всех элементах контейнера в том же формате.

11. Освободить память, удалив все динамически созданные объекты.

1.1 Описание входных данных

```
3
NAS1 100 500
NAS2 40 1000
NAS3 150 750
80
```

1.2 Описание выходных данных

```
Initial state
NAS1 100 500
NAS2 40 1000
```

```
NAS3 150 750
Empty pointer
Modified state
NAS1 100 500
NAS2 160 1000
NAS3 150 750
Empty pointer
```

2 МЕТОД РЕШЕНИЯ

Для решения задачи используется:

- объект objects класса vector<NetworkStorage*> предназначен для хранения указателей на динамически созданные объекты сетевого хранилища;
- объект obj класса NetworkStorage* предназначен для доступа к данным утсройства или его модификации;
- функция push_back() для добавления нового элемента в конец контейнера vector;
- функция size() для для получения текущего количества элементов в контейнере ;
- функция main для основная функция программы;
- динамический контейнер std::vector (библиотека vector).

Класс NetworkDevice:

- свойства/поля:
    - поле скорость передачи данных:
        - наименование — speed;
        - тип — int;
        - модификатор доступа — private;
- функционал:
    - метод NetworkDevice — конструтор для инициализации начальной скорости объекта;
    - метод setSpeed — сеттер для изменения значения скорости;
    - метод getSpeed — геттер для получения текущего значения скорости.

Класс NetworkStorage:

- свойства/поля:
    - поле название хранилища:
        - наименование — name;
        - тип — строкоывй;
        - модификатор доступа — private;
- функционал:
    - метод NetworkStorage — конструктор для инициализации имени и вызова конструкторов родительских классов;
    - метод getName — геттер для получения названия устройства;
    - метод printInfo — для вывода полной информации об объекте в консоль.

Класс StorageDevice:

- свойства/поля:
    - поле ёмкость памяти:
        - наименование — capacity;
        - тип — int;
        - модификатор доступа — private;
- функционал:
    - метод StorageDevice — конструктор для инициализации ёмкости памяти;
    - метод SetCapacity — сеттер для изменения значения ёмкости;
    - метод getCapacity — геттер для получения текущего значения ёмкости.

Таблица 1 – Иерархия наследования классов

№	Имя класса	Классы-наследники	Модификатор доступа при наследовании	Описание	Номер
1	NetworkDev			Родительский класс, предназначен для хранения и управления	

№	Имя класса	Классы-наследники	Модификатор доступа при наследовании	Описание	Номер
	ice			характеристиками скорости сетевого устройства	
		NetworkStorage	public		2
2	NetworkStorage			Дочерний класс, представляющий сетевое хранилище. Объединяет свойства сетевого узла и устройства хранения данных	
3	StorageDevice			Родительский класс для описания характеристик накопителя данных	
		NetworkStorage	public		2

3 ОПИСАНИЕ АЛГОРИТМОВ

Согласно этапам разработки, после определения необходимого инструментария в разделе «Метод», составляются подробные описания алгоритмов для методов классов и функций.

3.1 Алгоритм конструктора класса NetworkDevice

Функционал: Конструтор для инициализации начальной скорости объекта.

Параметры: целочисленный s.

Алгоритм конструктора представлен в таблице 2.

Таблица 2 – Алгоритм конструктора класса NetworkDevice

№	Предикат	Действия	№ перехода
1		Инициализировать поле speed значением параметра s	∅

3.2 Алгоритм метода setSpeed класса NetworkDevice

Функционал: сеттер для изменения значения скорости.

Параметры: целочисленный s.

Возвращаемое значение: void, метод ничего не возращает.

Алгоритм метода представлен в таблице 3.

Таблица 3 – Алгоритм метода setSpeed класса NetworkDevice

№	Предикат	Действия	№ перехода
1		Присвоить полю speed значение параметра s	∅

3.3 Алгоритм метода getSpeed класса NetworkDevice

Функционал: геттер для получения текущего значения скорости.

Параметры: нет.

Возвращаемое значение: целочисленный, возращает значение скорости.

Алгоритм метода представлен в таблице 4.

Таблица 4 – Алгоритм метода getSpeed класса NetworkDevice

№	Предикат	Действия	№ перехода
1		вернуть значение поля speed	∅

3.4 Алгоритм конструктора класса NetworkStorage

Функционал: Конструктор для инициализации имени и вызова конструкторов родительских классов.

Параметры: строковый n, целочисленный n, целочисленный c.

Алгоритм конструктора представлен в таблице 5.

Таблица 5 – Алгоритм конструктора класса NetworkStorage

№	Предикат	Действия	№ перехода
1		Вызвать конструтор родительского класса NetworkDevice с параметром s. Вызвать конструтор родительского класса StorageDevice с параметром c. Инициализировать поле name значением параметра n	∅

3.5 Алгоритм метода getName класса NetworkStorage

Функционал: Геттер для получения названия устройства.

Параметры: нет.

Возвращаемое значение: строковый, возращает строку с именем устройства.

Алгоритм метода представлен в таблице 6.

Таблица 6 – Алгоритм метода getName класса NetworkStorage

№	Предикат	Действия	№ перехода
1		Вернуть значение поля name	∅

3.6 Алгоритм метода printInfo класса NetworkStorage

Функционал: Для вывода полной информации об объекте в консоль.

Параметры: нет.

Возвращаемое значение: void, метод ничего не возращает.

Алгоритм метода представлен в таблице 7.

Таблица 7 – Алгоритм метода printInfo класса NetworkStorage

№	Предикат	Действия	№ перехода
1		Вывод: name << " " << getSpeed() << " " << getCapacity() << std::endl;	∅

3.7 Алгоритм конструктора класса StorageDevice

Функционал: Конструктор для инициализации ёмкости памяти.

Параметры: целочисленный c.

Алгоритм конструктора представлен в таблице 8.

Таблица 8 – Алгоритм конструктора класса StorageDevice

№	Предикат	Действия	№ перехода
1		Инициализировать поле capacity значением параметра c	∅

3.8 Алгоритм метода SetCapacity класса StorageDevice

Функционал: Сеттер для изменения значения ёмкости.

Параметры: целочисленный c.

Возвращаемое значение: void, метод нечего не возвращает.

Алгоритм метода представлен в таблице 9.

Таблица 9 – Алгоритм метода SetCapacity класса StorageDevice

№	Предикат	Действия	№ перехода
1		Присвоить полю capacity значение параметра c	∅

3.9 Алгоритм метода getCapacity класса StorageDevice

Функционал: Геттер для получения текущего значения ёмкости.

Параметры: нет.

Возвращаемое значение: целочисленный, возращает ёмкость памяти.

Алгоритм метода представлен в таблице 10.

Таблица 10 – Алгоритм метода getCapacity класса StorageDevice

№	Предикат	Действия	№ перехода
1		Вернуть значение поля capacity	∅

3.10 Алгоритм функции main

Функционал: основная функция программы.

Параметры: нет.

Возвращаемое значение: целочисленный, код возврата.

Алгоритм функции представлен в таблице 11.

Таблица 11 – Алгоритм функции main

№	Предикат	Действия	№ перехода
1		Объявление целочисленной переменной int n	2
2		Ввод значения количества элементов cin >> n	3
3		Создание вектора указателей	4

№	Предикат	Действия	№ перехода
		vector<NetworkStorage*> objects	
4		Инициализация счётчика цикла int i = 0	5
5	i < n		6
			11
6		Объявление строки name	7
7		Объявление целочисленных переменных speed и capacity	8
8		Ввод name, speed, capacity	9
9		Динамическое выделение памяти new NetworkStorage с параметрами name, speed, capacity и добавление указателя в вектор objects.push_back	10
10		увеличение счётчика i на 1	5
11		Добавление пустого указателя objects.push_back(nullptr) в конец вектора	12
12		Вывод текстового заголовка cout << "Initial state" << endl	13
13		инициализация счётчика цикла int i = 0	14
14	i < objects.size()		15
			19
15	objects[i] != nullptr		16
			17
16		Вызов метода отображения данных objects[i]->printInfo	18
17		Вывод сообщения cout << "Empty pointer" при пустом адресе	18
18		увеличение счётчика i на 1	14
19		Объявление целочисленной переменной min_speed	20
20		Ввод значения переменной min_speed	21

№	Предикат	Действия	№ перехода
21		инициализация счётчика цикла int i = 0	22
22	i < objects.size()		23
			31
23	object[i] == nullptr		24
			25
24		continue: пропуск пустых элементов	22
25	objects[i]->getSpeed() < min_speed		26
			30
26		Получение и сохранение текущего имени string curName = objects[i]->getName()	27
27		Получение и сохранение текущей ёмкости int curCapacity = objects[i]-> getCapacity()	28
28		Удаление объекта по указателю i-го элемента вектора objects	29
29		Создание нового объекта objects[i] = new NetworkStorage(curName, 2 * min_speed, curCapacity) с удвоенной скоростью и запись его адреса в вектор	30
30		увеличение счётчика i на 1	22
31		Вывод заголовка изменений cout << "Modified state" << endl	32
32		инициализация счётчика цикла int i = 0	33
33	i < objects.size()		34
			38
34	objects[i] != nullptr		35
			36
35		Вызов метода отображения данных objects[i]-	37

№	Предикат	Действия	№ перехода
		>printInfo	
36		Вывод сообщения cout << "Empty pointer" при пустом адресе	37
37		увеличение счётчика i на 1	34
38		инициализация счётчика цикла int i = 0	39
39	i < objects.size()		40
			43
40	objects[i] != nullptr		41
			42
41		Удаление объекта по указателю i-го элемента вектора objects	42
42		увеличение счётчика i на 1	40
43		возврат 0	∅

4 БЛОК-СХЕМЫ АЛГОРИТМОВ

Представим описание алгоритмов в графическом виде на рисунках 1-6.

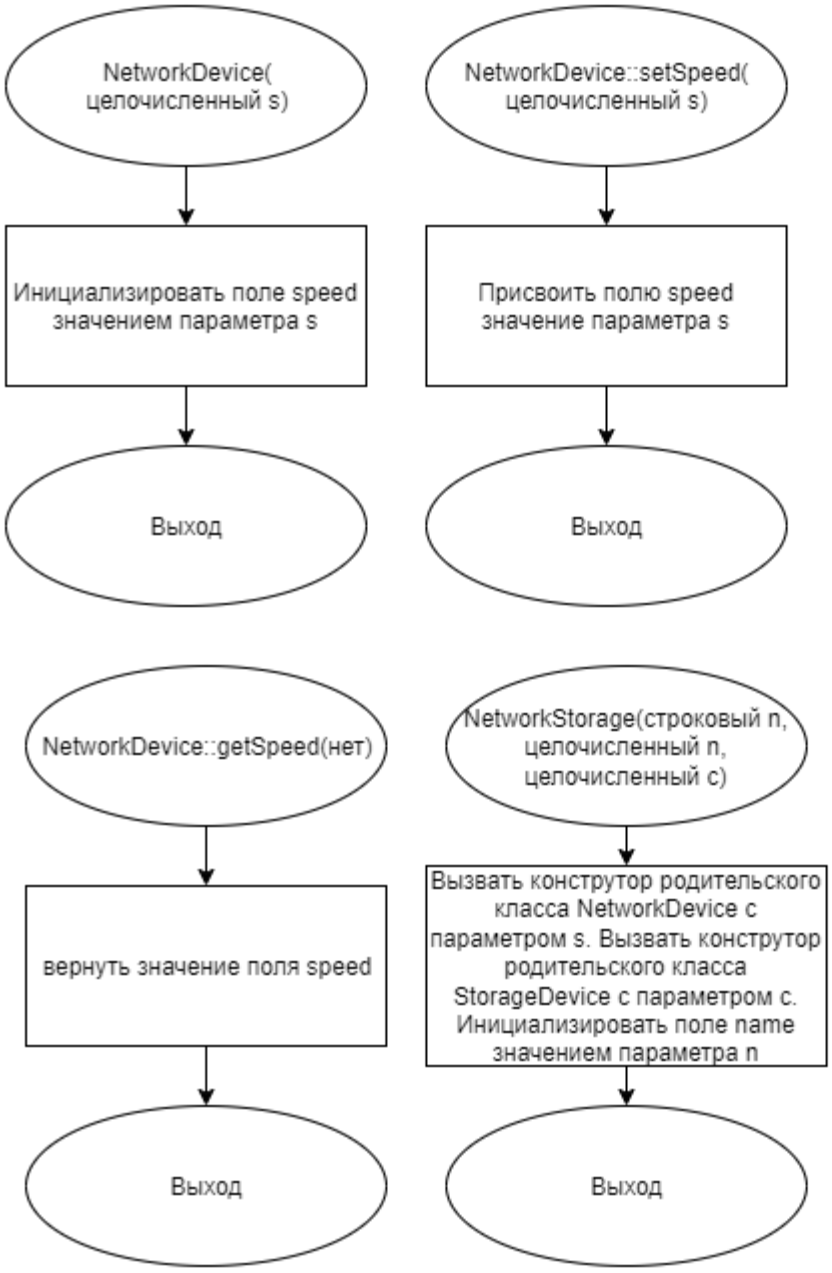

Рисунок 1 – Блок-схема алгоритма

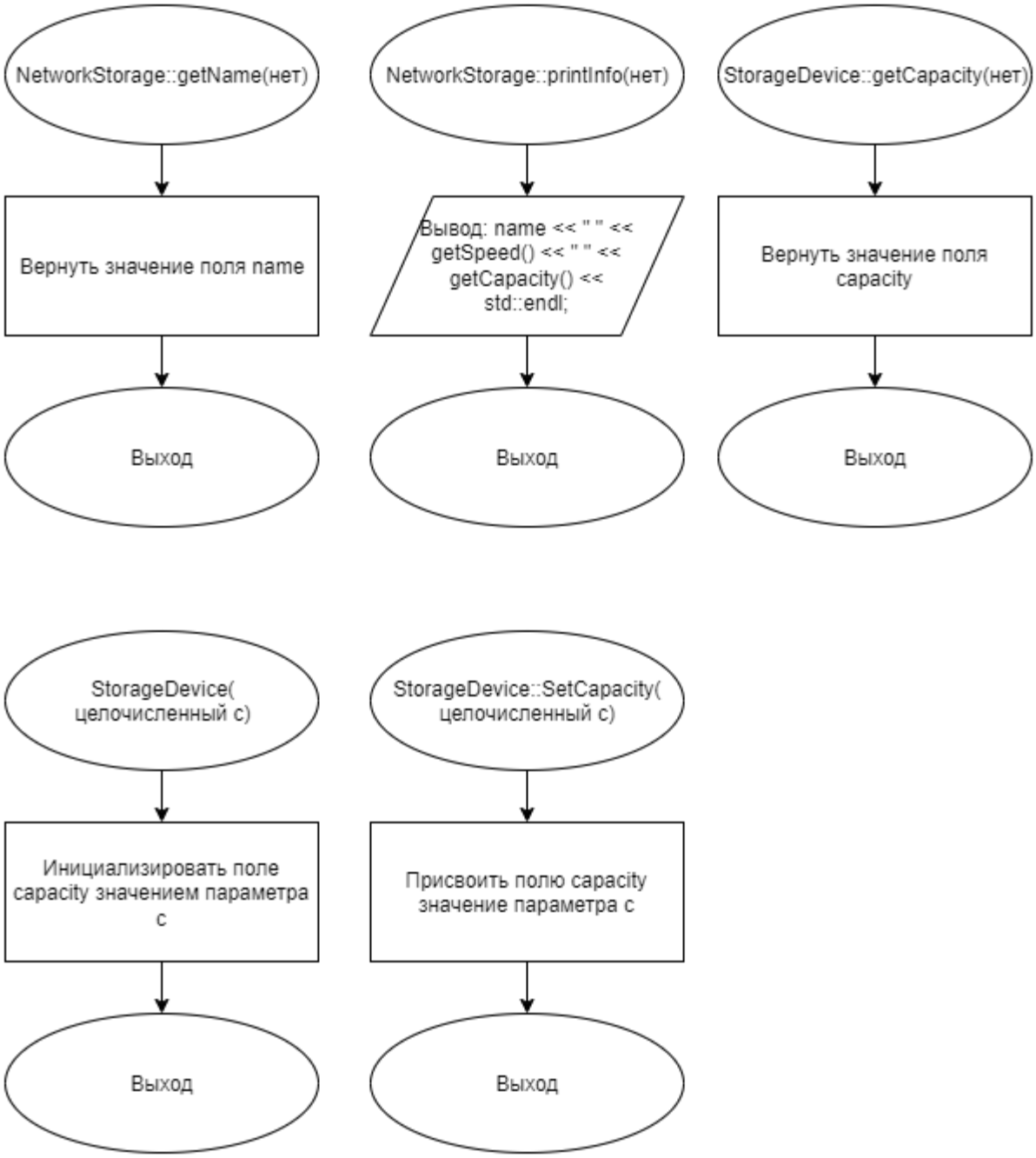

Рисунок 2 – Блок-схема алгоритма

Рисунок 3 – Блок-схема алгоритма

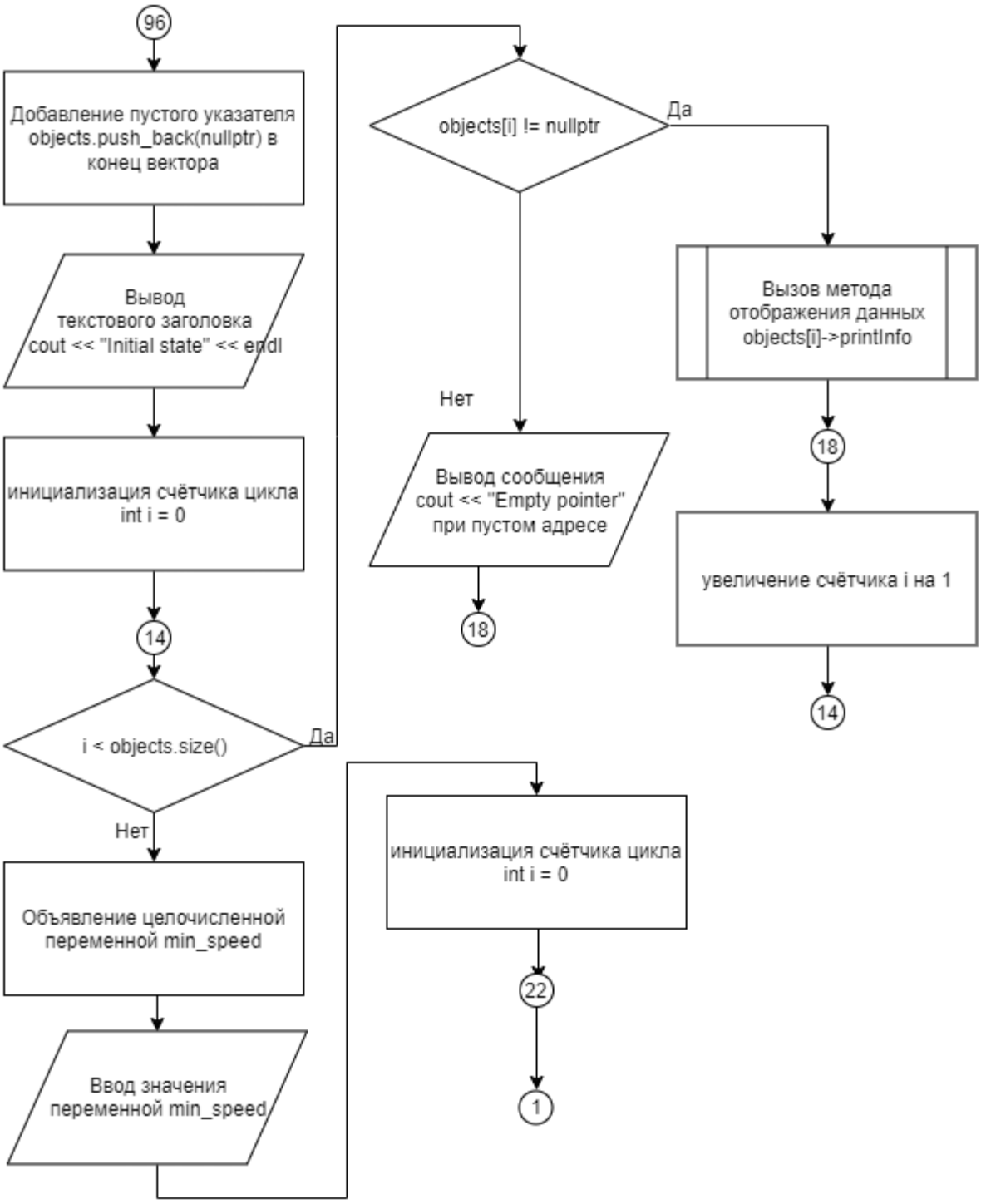

Рисунок 4 – Блок-схема алгоритма

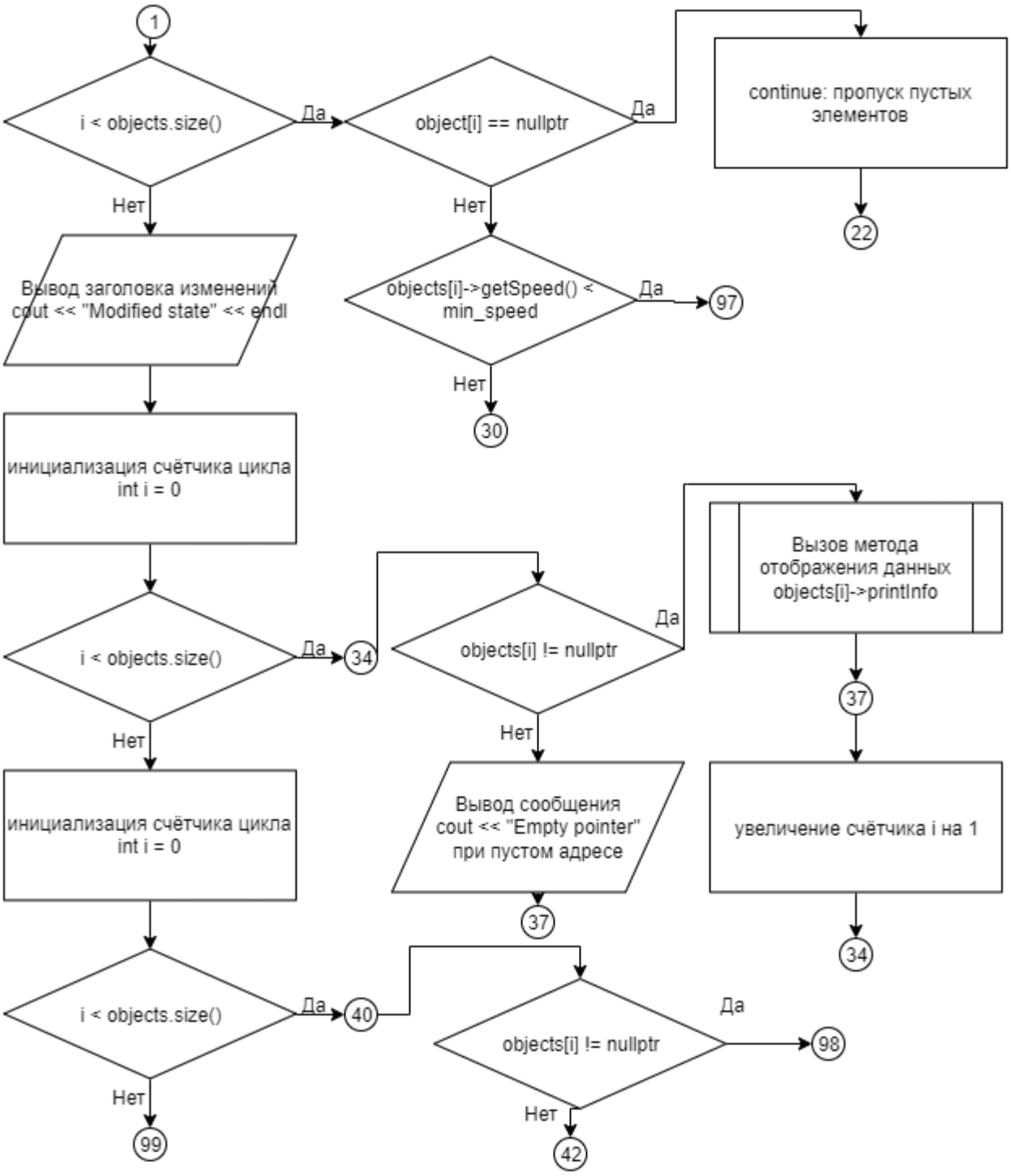

Рисунок 5 – Блок-схема алгоритма

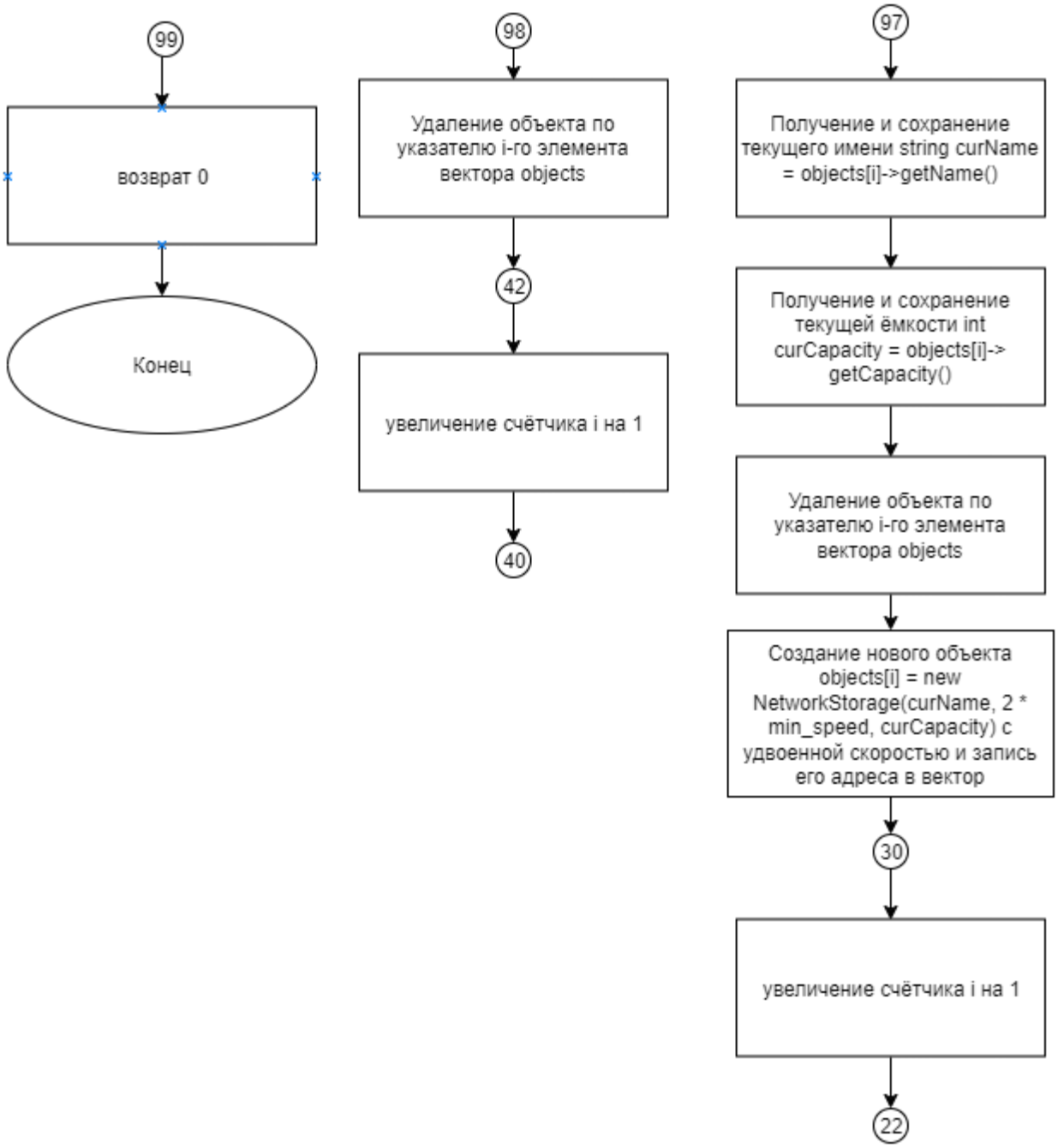

Рисунок 6 – Блок-схема алгоритма

5 КОД ПРОГРАММЫ

Программная реализация алгоритмов для решения задачи представлена ниже.

5.1 Файл main.cpp

Листинг 1 – main.cpp

```
#include <iostream>
#include <vector>
#include <string>
#include "NetworkStorage.h"

using namespace std;

int main()
{
   int n;
   cin >> n;

   vector<NetworkStorage*> objects;

   for (int i = 0; i < n; ++i){
      string name;
      int speed, capacity;
      cin >> name >> speed >> capacity;
      objects.push_back(new NetworkStorage(name, speed, capacity));
   }

   objects.push_back(nullptr);

   cout << "Initial state" << endl;

   for (int i = 0; i < objects.size(); ++i){
      if(objects[i] != nullptr){
         objects[i]-> printInfo();
      }
      else{
         cout << "Empty pointer" << endl;
      }
   }

   int min_speed;
   cin >> min_speed;

   for(int i = 0; i < objects.size(); ++i){
      if(objects[i] == nullptr) continue;
```

```
        if(objects[i] -> getSpeed() < min_speed){
           string curName = objects[i] -> getName();
           int curCapacity = objects[i] -> getCapacity();

           delete objects[i];
           objects[i]   =   new   NetworkStorage(curName,   2   *   min_speed,
curCapacity);
        }
     }

     cout << "Modified state" << endl;

     for (int i = 0; i < objects.size(); ++i){
        if(objects[i] != nullptr){
           objects[i]-> printInfo();
        }
        else{
           cout << "Empty pointer" << endl;
        }
     }

     for (int i = 0; i < objects.size(); ++i){
        if(objects[i] != nullptr){
           delete objects[i];
        }
     }

     return(0);
  }
```

5.2 Файл NetworkDevice.cpp

Листинг 2 – NetworkDevice.cpp

```
#include "NetworkDevice.h"

NetworkDevice::NetworkDevice(int s) : speed(s){}

void NetworkDevice::setSpeed(int s){
   speed = s;
}

int NetworkDevice::getSpeed() const{
   return speed;
}
```

5.3 Файл NetworkDevice.h

Листинг 3 – NetworkDevice.h

```
#ifndef __NETWORKDEVICE__H
#define __NETWORKDEVICE__H

class NetworkDevice{
private:
   int speed;

public:
   NetworkDevice(int s);
   void setSpeed(int s);
   int getSpeed() const;

};

#endif
```

5.4 Файл NetworkStorage.cpp

Листинг 4 – NetworkStorage.cpp

```
#include "NetworkStorage.h"
#include <iostream>

NetworkStorage::NetworkStorage(std::string n, int s, int c)
   :NetworkDevice(s), StorageDevice(c), name(n){}

std::string NetworkStorage:: getName() const{
   return name;
}

void NetworkStorage::printInfo() const{
   std::cout << name << " " << getSpeed() << " " << getCapacity() <<
std::endl;
}
```

5.5 Файл NetworkStorage.h

Листинг 5 – NetworkStorage.h

```
#ifndef __NETWORKSTORAGE__H
```

```
#define __NETWORKSTORAGE__H
#include "NetworkDevice.h"
#include "StorageDevice.h"
#include <string>

class NetworkStorage : public NetworkDevice, public StorageDevice{
private:
   std::string name;

public:
   NetworkStorage(std::string n, int s, int c);
   std::string getName() const;
   void printInfo() const;

};

#endif
```

5.6 Файл StorageDevice.cpp

Листинг 6 – StorageDevice.cpp

```
#include "StorageDevice.h"

StorageDevice::StorageDevice(int c): capacity(c){}

void StorageDevice::setCapacity(int c){
   capacity = c;
}

int StorageDevice::getCapacity() const{
   return capacity;
}
```

5.7 Файл StorageDevice.h

Листинг 7 – StorageDevice.h

```
#ifndef __STORAGEDEVICE__H
#define __STORAGEDEVICE__H

class StorageDevice{
private:
   int capacity;
```

```
public:
   StorageDevice(int C);
   void setCapacity(int C);
   int getCapacity() const;

};

#endif
```

6 ТЕСТИРОВАНИЕ

Результат тестирования программы представлен в таблице 12.

Таблица 12 – Результат тестирования программы

Входные данные	**Ожидаемые выходные данные**	**Фактические выходные данные**
3 NAS1 100 500 NAS2 40 1000 NAS3 150 750 80	Initial state NAS1 100 500 NAS2 40 1000 NAS3 150 750 Empty pointer Modified state NAS1 100 500 NAS2 160 1000 NAS3 150 750 Empty pointer	Initial state NAS1 100 500 NAS2 40 1000 NAS3 150 750 Empty pointer Modified state NAS1 100 500 NAS2 160 1000 NAS3 150 750 Empty pointer

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19 Единая система программной документации.

2. Методическое пособие студента для выполнения практических заданий, контрольных и курсовых работ по дисциплине «Объектно-ориентированное программирование» [Электронный ресурс] – URL: https://mirea.aco-avrora.ru/student/files/methodichescoe_posobie_dlya_laboratornyh_rabot_3.pdf (дата обращения 05.05.2021).

3. Приложение к методическому пособию студента по выполнению заданий в рамках курса «Объектно-ориентированное программирование» [Электронный ресурс]. URL: https://mirea.aco-avrora.ru/student/files/Prilozheniye_k_methodichke.pdf (дата обращения 05.05.2021).

4. Шилдт Г. C++: базовый курс. 3-е изд. Пер. с англ.. — М.: Вильямс, 2019. — 624 с.

5. Видео лекции по курсу «Объектно-ориентированное программирование» [Электронный ресурс]. АСО «Аврора».

6. Антик М.И. Дискретная математика [Электронный ресурс]: Учебное пособие /Антик М.И., Казанцева Л.В. — М.: МИРЭА — Российский технологический университет, 2018 — 1 электрон. опт. диск (CD-ROM).